LAPRAK IOT

Praktikum 3



Oleh:

NAMA : **M. Kurnia Akbar**
NPM : **25782101**
KELAS : **TRI – 3D**

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T
Agus Ambarwari, S.Pd., M.Kom.

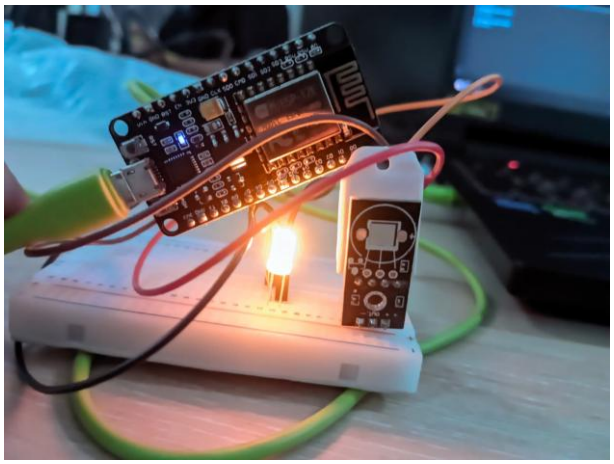
TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2025/2026

Praktikum 1:
NodeMCU sebagai *Web Server* (Menyajikan HTML)

Tujuan

Merakit NodeMCU sebagai server pintar. Modul ini membaca sensor DHT dan menampilkan suhunya di sebuah halaman web (UI), sekaligus menyediakan tombol di web tersebut untuk menghidupkan/mematikan lampu LED.

Rangkaian





Langkah Kerja Rangkaian:

1. Hubungkan pin DATA / OUT / S DHT ke pin D4 (GPIO 2). (Pastikan VCC DHT ke 3V3, GND ke GND).
2. Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12). Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke pin GND NodeMCU. *(Catatan: LED ini dikendalikan oleh pin D6 / GPIO 12 yang pada kode program didefinisikan sebagai relayPin).*

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*). Tuliskan kode program di bawah. **Jangan lupa mengganti variabel ssid dan password!**
2. *Catatan teknis:* Kita menggunakan teknik **Raw String Literal** C++ (R"rawliteral(...)rawliteral") untuk menulis kode antarmuka HTML multibaris, sehingga kita tidak repot mengetikkan tanda kutip "" di setiap baris HTML

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <DHT.h>

const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

ESP8266WebServer server(80);

const byte dhtPin = 2;
const byte relayPin = 12;
DHT dht(dhtPin, DHT11);

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>IoT Dashboard</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 50px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer;}
    .btn-on { background-color: #4CAF50; color: white; border: none; }
    .btn-off { background-color: #f44336; color: white; border: none; }
    .sensor-box { font-size: 24px; font-weight: bold; }
  </style>
</head>
<body>
  <h1>ESP8266 Web Server</h1>
  <div class="sensor-box">
    <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
  </div>
  <h2>Kendali Relay</h2>
  <a href="/relay/on"><button class="btn-on">ON</button></a>
  <a href="/relay/off"><button class="btn-off">OFF</button></a>
</body>
</html>
)rawliteral";

void handleRoot() {
  String html = index_html; // Salin kerangka HTML ke variabel dinamis
  float t = dht.readTemperature();
```

```
  // Mengganti teks placeholder %TEMPERATURE% dengan suhu nyata
  if (isnan(t)) {
    html.replace("%TEMPERATURE%", "--");
  } else {
    html.replace("%TEMPERATURE%", String(t));
  }

  server.send(200, "text/html", html);
}

void handleRelayOn() {
  digitalWrite(relayPin, HIGH);
  server.sendHeader("Location", "/");
  server.send(303);
}

void handleRelayOff() {
  digitalWrite(relayPin, LOW);
  server.sendHeader("Location", "/");
  server.send(303);
}

void setup() {
  Serial.begin(115200);
  pinMode(relayPin, OUTPUT);
  digitalWrite(relayPin, LOW);
  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nIP Address Server Anda: ");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.on("/relay/on", handleRelayOn);
  server.on("/relay/off", handleRelayOff);
  server.begin();
}

void loop() {
  server.handleClient();
}
```

3. **Upload** kode tersebut. Buka **Serial Monitor**, dan catat alamat IP yang didapat (contoh: 192.168.43.15).
4. Hubungkan Laptop/HP Anda ke Wi-Fi yang sama, buka *browser* (Chrome/Safari), lalu ketikkan alamat IP tersebut. Klik tombol ON dan OFF di layar untuk melihat LED menyala dan padam.

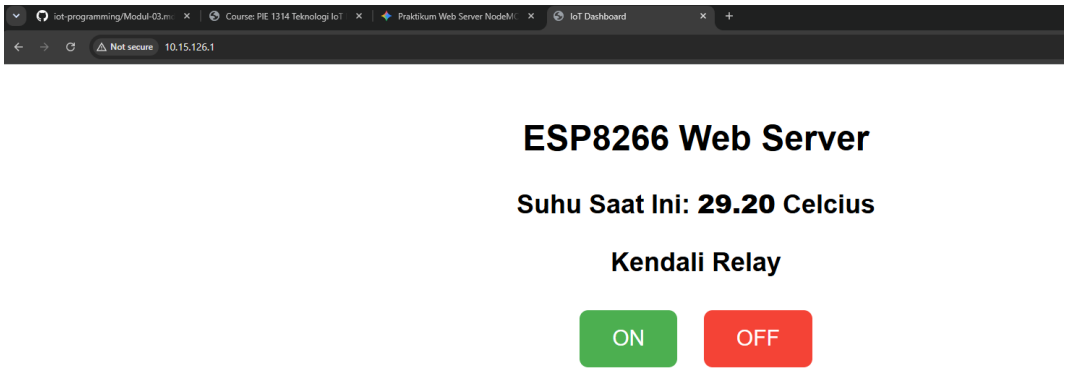
Penjelasan Singkat Kode:

- ESP8266WebServer server(80);: Menginisialisasi *library Server* agar terus mendengarkan panggilan di jalur lalu lintas standar web (*Port 80*).
- html.replace("%TEMPERATURE%", String(t));: Fitur manipulasi teks (*String*). Perintah ini secara cerdas akan menyapu dokumen HTML, mencari teks *placeholder* %TEMPERATURE%, dan menggantinya dengan variabel angka suhu yang didapat dari sensor aktual sebelum dikirim ke layar pengunjung. Inilah esensi dari halaman *web* dinamis.
- server.on("/relay/on", handleRelayOn);: Fungsi *Routing*. Jika ada klien mengunjungi tautan URL berakhiran /relay/on, server akan menjalankan aksi mengubah *state* LED (menyalakan/mematikan output), lalu me-

redirect layar pengunjung kembali ke halaman depan (Location: /) via kode 303.

- server.handleClient();: Berada di dalam loop() untuk terus-menerus mengecek apakah ada permintaan pengunjung yang datang setiap milidetiknya.

Hasil Pengujian:

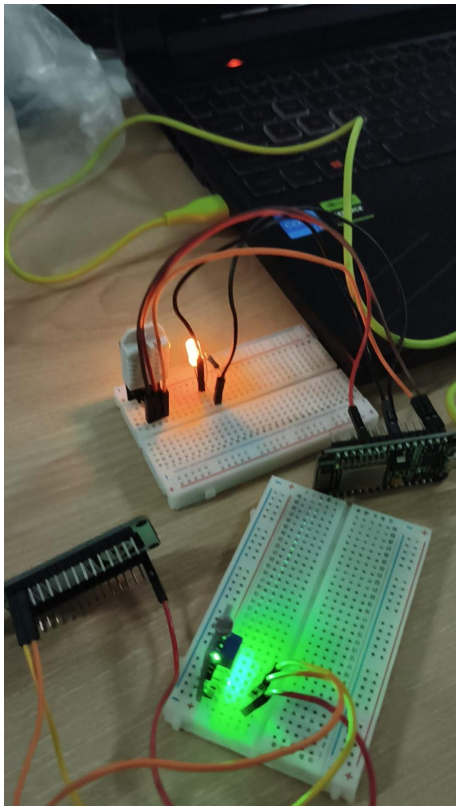


Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)

Tujuan

Memprogram NodeMCU kedua (atau meminta teman kelompok di sebelah Anda) untuk menjadi *Client*. Perangkat Klien ini dipasangi LDR. Jika LDR tertutup kegelapan, ia akan mengirim sinyal HTTP secara jarak jauh (via jaringan Wi-Fi) untuk menyalakan LED di *Server* milik teman Anda (Praktikum 1).

Rangkaian



Langkah Kerja Rangkaian:

1. Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin A0.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*).
2. Tuliskan kode program di bawah. **Ganti variabel serverName dengan IP Address NodeMCU Praktikum 1 milik teman sebelah Anda.** (Pastikan Anda menggunakan Wi-Fi yang sama persis).

```ino
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>

const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

const char* serverName = "http://192.168.1.15/relay/on";
const byte ldrPin = A0;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nClient Terhubung ke Wi-Fi!");
}

void loop() {
  int ldrValue = analogRead(ldrPin);

  if ((WiFi.status() == WL_CONNECTED) && (ldrValue < 300)) {
    WiFiClient client;
    HTTPClient http;

    http.begin(client, serverName);
    int httpResponseCode = http.GET();

    Serial.print("HTTP Response code: ");
    Serial.println(httpResponseCode);

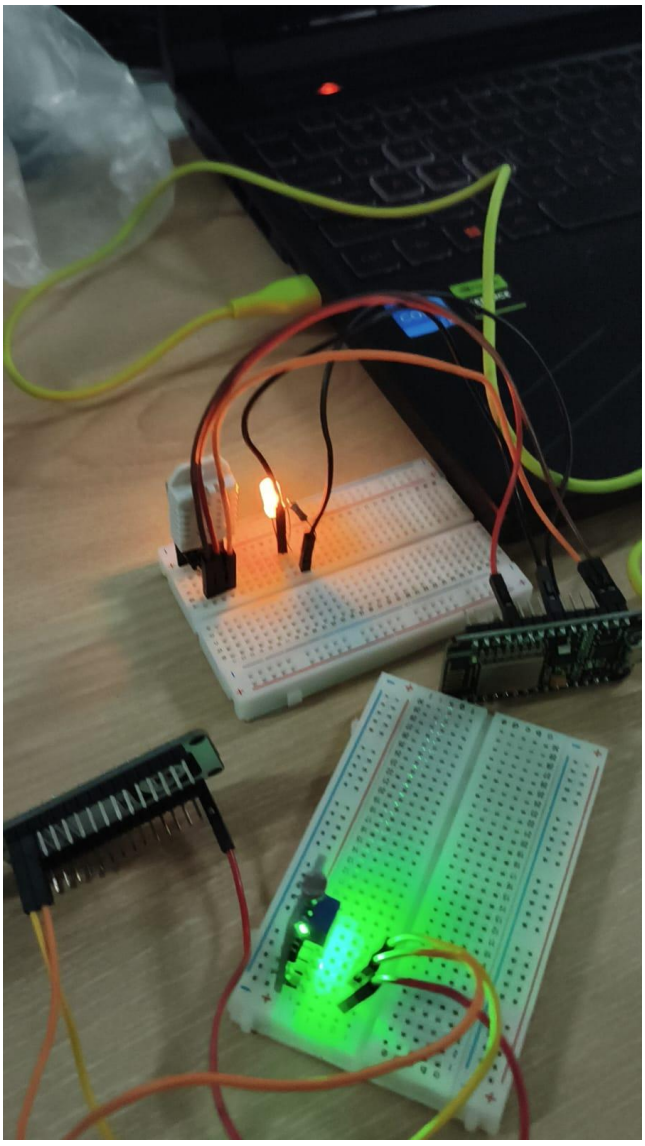
    http.end();
    delay(10000);
  }
  delay(2000);
}
```

3. **Upload** kode tersebut, dan buka Serial Monitor. Saat LDR Anda di *Client* ditutup tangan hingga gelap, perhatikan **LED** di NodeMCU *Server* milik teman Anda pasti akan menyala.

Penjelasan Singkat Kode:

- WiFiClient client; dan HTTPClient http;: Kelas pemanggil untuk memulai eksekusi permintaan *client* jarak jauh (mirip fungsi aplikasi *browser* di laptop).
- http.begin(client, serverName);: Menargetkan sasaran URL *server*.
- int httpResponseCode = http.GET();: Mengeksekusi ketukan pintu (*Request* HTTP GET). Jika Server merespons sukses, nilai variabel ini akan berisi angka standar jaringan seperti 200 (OK) atau 303 (Redirect). Jika URL rusak atau *Server offline*, variabel akan mereturn nilai negatif.
- delay(10000);: *Cooldown* panjang 10 detik ini ditambahkan agar *Client* tidak memborbardir (*flood/DDoS*) Server dengan permintaan on secara terus-menerus setiap detik ketika kondisinya sedang gelap.

Hasil Pengujian



Latihan

Latihan 1 (Pemahaman Jaringan): Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

Mode Station (STA) digunakan pada praktikum web server ESP8266 dengan pertimbangan logis berikut:

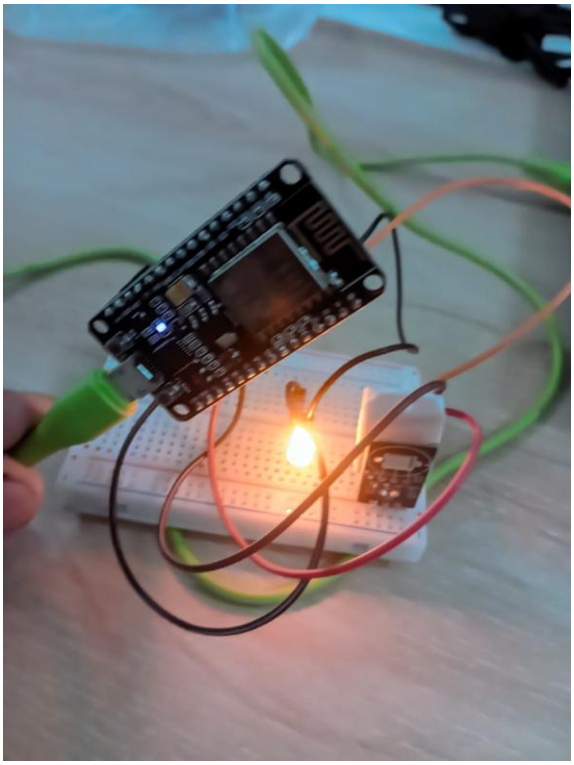
- Aksesibilitas dan Konektivitas Internet: Pada mode STA, ESP8266 bertindak sebagai *client* yang terhubung ke jaringan Wi-Fi lokal (seperti *router* rumah atau *hotspot* HP). Hal ini memungkinkan ESP8266 terhubung ke internet untuk kebutuhan integrasi cloud, pengiriman data telemetri, sinkronisasi waktu (NTP), atau pengiriman notifikasi di masa mendatang.
- Kemudahan Akses Perangkat: Pengguna dapat mengakses *dashboard* web server ESP8266 dari perangkat apa pun (laptop, HP, tablet) yang terhubung ke jaringan Wi-Fi lokal yang sama tanpa perlu berganti atau terputus dari koneksi internet utama.
- Efisiensi dan Efisiensi Daya: Mode Access Point (AP) membuat ESP8266 harus memancarkan sinyal Wi-Fi sendiri. Mode AP membatasi jumlah koneksi *client* secara drastis, mengonsumsi lebih banyak daya, dan mengisolasi jaringan sehingga perangkat pengontrol kehilangan akses internet saat terhubung ke ESP8266.

Latihan 2 (Modifikasi Antarmuka & Telemetri): Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:

i. Menampilkan Kelembapan: Tambahkan pembacaan kelembapan menggunakan fungsi dht.readHumidity(). Sisipkan placeholder %HUMIDITY% pada kerangka HTML di dalam tag (contoh: Kelembapan: %HUMIDITY% %), lalu perbarui fungsi handleRoot() menggunakan html.replace("%HUMIDITY%", ...) agar nilai kelembapan muncul di halaman *web*.

ii. Tombol Dinamis Tunggal (*Toggle Button*): Ubah antarmuka agar hanya menggunakan 1 buah tombol tunggal yang statusnya adaptif mengikuti kondisi LED saat ini:

   - Jika LED sedang OFF, tombol menampilkan tulisan "NYALAKAN (ON)" (misal: warna hijau/biru) dan mengarahkan aksi ke /relay/on.
   - Jika LED sedang ON, tombol otomatis berubah menjadi tulisan "MATIKAN (OFF)" (warna merah) dan mengarahkan aksi ke /relay/off.
     (Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti %RELAY_BUTTON%, lalu di fungsi handleRoot(), periksa status pin relay/LED menggunakan digitalRead(relayPin) sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai).



Langkah Kerja Pemrograman:

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <DHT.h>

const char* ssid = "Redmi-12";
const char* password = "22345678";

ESP8266WebServer server(80);

const byte dhtPin = 2;
const byte relayPin = 12;
DHT dht(dhtPin, DHT22);

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta http-equiv="refresh" content="5">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>IoT Dashboard</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 50px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer;}
    .btn-on { background-color: #4CAF50; color: white; border: none; }
    .btn-off { background-color: #f44336; color: white; border: none; }
    .sensor-box { font-size: 24px; font-weight: bold; }
  </style>
</head>
<body>
  <h1>ESP8266 Web Server</h1>
  <div class="sensor-box">
    <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
    <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
  </div>
  <h2>Kendali Relay</h2>
  %RELAY_BUTTON%
  </div>
</body>
</html>
)rawliteral";

void handleRoot() {
  String html = index_html;

  float t = dht.readTemperature();
  float h = dht.readHumidity();

  // Menampilkan suhu
  if (isnan(t)) {
    html.replace("%TEMPERATURE%", "--");
  } else {
    html.replace("%TEMPERATURE%", String(t));
  }

  // Menampilkan kelembapan
  if (isnan(h)) {
    html.replace("%HUMIDITY%", "--");
  } else {
    html.replace("%HUMIDITY%", String(h));
  }

  // Membuat tombol relay dinamis
  String relayButton;

  if (digitalRead(relayPin) == LOW) {
    relayButton = "<a href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN (ON)</button></a>";
  } else {
    relayButton = "<a href=\"/relay/off\"><button class=\"btn-off\">MATIKAN (OFF)</button></a>";
  }

  html.replace("%RELAY_BUTTON%", relayButton);

  server.send(200, "text/html", html);
}

void handleRelayOn() {
  digitalWrite(relayPin, HIGH);
  server.sendHeader("Location", "/");
  server.send(303);
}

void handleRelayOff() {
  digitalWrite(relayPin, LOW);
  server.sendHeader("Location", "/");
  server.send(303);
}

void setup() {
  Serial.begin(115200);
  pinMode(relayPin, OUTPUT);
  digitalWrite(relayPin, LOW);
  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nIP Address Server Anda: ");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.on("/relay/on", handleRelayOn);
  server.on("/relay/off", handleRelayOff);
  server.begin();
}

void loop() {
  server.handleClient();
}
```

Hasil Pengujian :



Latihan 3 (Analisis Kode Client): Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

Saat NodeMCU Server mati (offline), perangkat Client yang mencoba melakukan koneksi ke alamat IP Server akan mengalami kegagalan pada lapisan jaringan (network connection failure) sebelum sempat menerima respons HTTP.

Berikut adalah hasil analisis yang akan dicetak pada Serial Monitor Client:

- HTTP Response Code: Akan mencetak nilai negatif, yaitu -1 (atau kode error internal HTTPC_ERROR_CONNECTION_REFUSED).
- Pesan / Respons: Fungsi http.GET() mengembalikan nilai -1 karena soket TCP gagal terhubung (*Connection refused* / *Timed out*). Kode HTTP standar (seperti 200, 404, atau 500) tidak akan pernah diterima karena tidak ada server aktif yang memproses header request.

Latihan 4 (Pengembangan Algoritma): Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) HANYA JIKA sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

Algoritma pengondisian pada fungsi loop dilakukan dengan membaca nilai suhu dari sensor DHT secara berkala, memvalidasi keberhasilan pembacaan, dan memeriksa apakah suhunya melebihi 35 °C; jika kondisi tersebut terpenuhi, klien akan mengirimkan permintaan HTTP GET ke server, namun jika tidak, proses pengiriman akan dilewati sebelum sistem menunggu jeda waktu untuk pembacaan berikutnya.

Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur

Tujuan

Mampu memanipulasi struktur kerangka HTML mentah yang tersimpan di memori NodeMCU untuk mencapai interaktivitas tampilan otomatis (*Auto-Refresh*).

Rangkaian



Langkah Kerja Program:

Instruksi:

1. Lakukan modifikasi kode hanya pada sisi NodeMCU Server (Latihan 2).
2. Modifikasi variabel Raw Literal index_html. Sisipkan perintah *Meta Refresh HTML* (sebuah Tag standar web kuno namun ampuh) persis di dalam cakupan *tag* <head> untuk memaksa *browser* memuat ulang halaman secara sekuensial setiap 5 detik.
3. *Hint (Petunjuk):* Sintaks standar yang harus disisipkan adalah <meta http-equiv="refresh" content="5">.
4. Setelah selesai, buka *browser*, dan amati layarnya. Biarkan selama 20 detik tanpa menyentuh *mouse/keyboard*.

```ino
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <DHT.h>

const char* ssid = "Redmi 12";
const char* password = "22345678";

ESP8266WebServer server(80);

const byte dhtPin = 2;
const byte relayPin = 12;
DHT dht(dhtPin, DHT22);

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>IoT Dashboard</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 50px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer;}
    .btn-on { background-color: #4CAF50; color: white; border: none; }
    .btn-off { background-color: #f44336; color: white; border: none; }
    .sensor-box { font-size: 24px; font-weight: bold; }
  </style>
</head>
<body>
  <h1>ESP8266 Web Server</h1>
  <div class="sensor-box">
    <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
  </div>
  <h2>Kendali Relay</h2>
  <a href="/relay/on"><button class="btn-on">ON</button></a>
  <a href="/relay/off"><button class="btn-off">OFF</button></a>
</body>
</html>
)rawliteral";

void handleRoot() {
  String html = index_html; // Salin kerangka HTML ke variabel dinamis
  float t = dht.readTemperature();

  // Mengganti teks placeholder %TEMPERATURE% dengan suhu nyata
  if (isnan(t)) {
    html.replace("%TEMPERATURE%", "--");
  } else {
    html.replace("%TEMPERATURE%", String(t));
  }

  server.send(200, "text/html", html);
}

void handleRelayOn() {
  digitalWrite(relayPin, HIGH);
  server.sendHeader("Location", "/");
  server.send(303);
}

void handleRelayOff() {
  digitalWrite(relayPin, LOW);
  server.sendHeader("Location", "/");
  server.send(303);
}
```

Hasil Pengujian

